

BLG462 - Büyük Veri Dönem Projesi

Twitter Verisi ile Uçtan Uca Büyük Veri Pipeline'ı Proje Raporu

Hazırlayan: Berat Turan

Öğrenci Numarası: 221201018

GİRİŞ VE PROJE AMACI

Bu projede Kaggle'dan indirdiğim "Twitter US Airline Sentiment" veri setini kullanarak baştan sona bir büyük veri pipeline'ı kurdum. Temel olarak şunları yaptım: ham tweet verisini Pandas ile temizledim, temiz veriyi MongoDB'ye yükleyip çeşitli sorgular yazdım, PySpark ile dağıtık işleme mantığını uyguladım ve son olarak veriden anlamlı grafikler çıkardım. Raporun geri kalanında bu adımların her birini kodlarıyla ve çıktılarıyla birlikte anlatıyorum.

AŞAMA 1: VERİ TEMİNİ VE KEŞİF

1.1. Veri Seti Hakkında

Kaggle'dan indirdiğim veri seti, ABD'deki havayolu şirketlerine atılmış yaklaşık 14.000 tweetten oluşuyor. Toplamda 15 sütun var: tweet metni, hangi havayoluna atıldığı, duygu durumu (pozitif/negatif/nötr), negatifse sebebi, güven skoru, kullanıcı adı, konum, tarih gibi bilgiler. Veri seti zaten etiketlenmiş durumda, yani her tweetin duygu durumu önceden belirlenmiş.

1.2. DIKW Piramidi Çerçevesinde Veri Setinin Değerlendirilmesi

Bu veri setini DIKW piramidine oturtmaya çalıştığımda şöyle bir tablo çıktı:

- **Data (Veri):** Elimdeki ham hali 14.000 satırlık bir CSV dosyası. İçinde tweet metinleri, tarihler, havayolu isimleri var ama düzenlenmemiş, temizlenmemiş bir yığın.
- **Information (Bilgi):** Bu ham veriye baktığımda tweetlerin pozitif, negatif ve nötr olarak etiketlendiğini görüyorum. Negatif olanların da "rötar", "kayıp bagaj", "müşteri hizmetleri" gibi alt kategorilere ayrıldığını fark ediyorum. Yani artık sadece sayı değil, anlam var.
- **Knowledge (Bilgi Birikimi):** Veriyi analiz ettiğimde mesela "United Airlines en çok müşteri hizmetleri konusunda şikayet alıyor" ya da "şikayetler belirli günlerde patlama yapıyor" gibi örüntüler ortaya çıkıyor. Bu artık karar vermeye yakın bir seviye.
- **Wisdom (Bilgelik):** Piramidin en tepesi. Burası artık "peki ne yapalım?" sorusunun cevabı. Mesela bir havayolu şirketi bu verilere bakarak müşteri hizmetleri ekibini güçlendirebilir ya da bagaj takip sistemini yenileyebilir. Yani veriden stratejik karara dönüşüm.

1.3. Veri Keşfi ve Temel İstatistikler

Pandas ile veri setini okuttuğumda 14640 satır ve 15 sütun olduğunu gördüm. `df.info()` ile baktığımda bazı sütunlarda ciddi miktarda eksik değer olduğunu fark ettim (özellikle `negative_reason`, `airline_sentiment_gold`, `tweet_coord`). `df.describe()` çıktısı da sayısal sütunların dağılımını gösterdi.

```
import pandas as pd
import re

# veri setini okuyorum, encoding latin-1 olmak zorunda yoksa turkce karakterlerde patlıyor
df = pd.read_csv('../data/raw/Tweets.csv', encoding='latin-1')

# boyutuna bakalım kac satir kac sutun var
print("Satir x Sutun:", df.shape)
print("\nSutun isimleri:\n", df.columns.tolist())

# ilk 5 kayda goz atalım
df.head()
```

Satir x Sutun: (14640, 15)

Sutun isimleri:

['tweet_id', 'airline_sentiment', 'airline_sentiment_confidence', 'negativereason', 'negativereason_confidence', 'airline', 'airline_sentiment_gold', 'name', 'negativereason_gold', 'retweet_count', 'text', 'tweet_coord', 'tweet_created', 'tweet_location', 'user_timezone']

	tweet_id	airline_sentiment	airline_sentiment_confidence	negativereason	negativereason_confidence	airline	airline_sentiment_gold	name	negativereason_gold	retweet_count	text	tweet_coord	tweet_created	tweet_location	user_timezone
0	570306133677760513	neutral	1.0000	NaN	NaN	Virgin America	NaN	cairdin	NaN	1	...	...	...	...	...
1	570301130888122368	positive	0.3486	NaN	0.0000	Virgin America	NaN	jnardino	NaN	1	...	...	...	...	...
2	570301083672813571	neutral	0.6837	NaN	NaN	Virgin America	NaN	yvonnalynn	NaN	1	...	...	...	...	...
3	570301031407624196	negative	1.0000	Bad Flight	0.7033	Virgin America	NaN	jnardino	NaN	1	...	...	...	...	...
4	570300817074462722	negative	1.0000	Can't Tell	1.0000	Virgin America	NaN	jnardino	NaN	1	...	...	...	...	...

```
print("--- Veri Tipleri ve Eksik Deger Durumu ---")
df.info()

print("\n--- Temel Istatistikler ---")
df.describe(include='all')
```

--- Veri Tipleri ve Eksik Deger Durumu ---

<class 'pandas.DataFrame'>

RangeIndex: 14640 entries, 0 to 14639

Data columns (total 15 columns):

#	Column	Non-Null Count	Dtype
0	tweet_id	14640 non-null	int64
1	airline_sentiment	14640 non-null	str
2	airline_sentiment_confidence	14640 non-null	float64
3	negativereason	9178 non-null	str
4	negativereason_confidence	10522 non-null	float64
5	airline	14640 non-null	str
6	airline_sentiment_gold	40 non-null	str
7	name	14640 non-null	str
8	negativereason_gold	32 non-null	str
9	retweet_count	14640 non-null	int64
10	text	14640 non-null	str
11	tweet_coord	1019 non-null	str
12	tweet_created	14640 non-null	str
13	tweet_location	9987 non-null	str
14	user_timezone	9820 non-null	str

dtypes: float64(2), int64(2), str(11)

memory usage: 1.7 MB

--- Temel Istatistikler ---

	tweet_id	airline_sentiment	airline_sentiment_confidence	negativereason	negativereason_confidence	airline	airline_sentiment_gold	name	negativereason_gold
count	1.464000e+04	14640	14640.000000	9178	10522.000000	14640	40	14640	32
unique	NaN	3	NaN	10	NaN	6	3	7701	3
top	NaN	negative	NaN	Customer Service Issue	NaN	United	negative	JetBlueNews	Customer Service Issue
freq	NaN	9178	NaN	2910	NaN	3822	32	63	2910
mean	5.692184e+17	NaN	0.900169	NaN	0.638298	NaN	NaN	NaN	NaN
std	7.791112e+14	NaN	0.162830	NaN	0.330440	NaN	NaN	NaN	NaN
min	5.675883e+17	NaN	0.335000	NaN	0.000000	NaN	NaN	NaN	NaN
25%	5.685592e+17	NaN	0.692300	NaN	0.360600	NaN	NaN	NaN	NaN
50%	5.694779e+17	NaN	1.000000	NaN	0.670600	NaN	NaN	NaN	NaN
75%	5.698905e+17	NaN	1.000000	NaN	1.000000	NaN	NaN	NaN	NaN
max	5.703106e+17	NaN	1.000000	NaN	1.000000	NaN	NaN	NaN	NaN

Şekil 1-2 : df.shape, df.info() ve df.describe() çıktıları

AŞAMA 2: VERİ TEMİZLEME VE HAZIRLIK

2.1. Temizleme İşlemleri

Veri setini analize hazır hale getirmek için birkaç adım uyguladım:

Önce tweet metinlerini temizledim. Regex kullanarak URL'leri, @mention'ları ve özel karakterleri sildim. Hashtag'lerin sadece # işaretini kaldırdım, kelimenin kendisini bıraktım çünkü analiz için işe yarayabilir. Son olarak metinleri küçük harfe çevirdim ve fazla boşlukları düzelttim.

Sonra duplicate kontrolü yaptım. `drop_duplicates()` ile birebir aynı olan satırları sildim. Bu adımda toplamda birkaç yüz kayıt çıktı.

Eksik değerler konusunda biraz düşündüm. İlk başta null olan satırları tamamen silmeyi düşündüm ama o zaman ciddi veri kaybı olurdu. Özellikle `negativereason` sütununda çok fazla boşluk vardı (çünkü pozitif ve nötr tweetlerde doğal olarak şikayet sebebi yok). Bunları silmek yerine "Belirtilmedi" olarak doldurdum. Diğer az kullanılan sütunlardaki boşlukları da "Bilinmiyor" yaptım. Böylece hiçbir satırı kaybetmeden devam edebildim.

Son olarak `tweet_created` sütunu string olarak geliyordu, bunu `pd.to_datetime()` ile düzgün tarih formatına çevirdim. Bu sayede sonra zaman serisi analizi yapabilecektim.

```
# temizleme öncesi kayıt sayısını not alıyorum sonra karşılaştırmak için
baslangic_kayit = len(df)
print(f"Temizleme öncesi toplam kayıt: {baslangic_kayit}")

def metin_temizle(text):
    """tweet metnini temizleyen fonksiyon"""
    if type(text) != str:
        return text

    # url leri sil (http ile başlayan herseyi)
    text = re.sub(r'http\S+|www\S+|https\S+', '', text)
    # @mention leri sil
    text = re.sub(r'@\w+', '', text)
    # hashtag isaretini sil ama kelimeyi bırak
    text = re.sub(r'#', '', text)
    # geriye kalan özel karakterleri temizle, sadece harf rakam bosluk kalsin
    text = re.sub(r'^\w\s', '', text)

    # fazla bosluklari duzelt ve kucuk harfe cevir
    text = " ".join(text.split()).lower()
    return text

# fonksiyonu text sutununa uygulayip yeni sutun olusturuyorum
df['cleaned_text'] = df['text'].apply(metin_temizle)

# temizlenmis haliyle eski halini yan yana gorelim
df[['text', 'cleaned_text']].head()
```

Temizleme öncesi toplam kayıt: 14640

	text	cleaned_text
0	@VirginAmerica What @dhepburn said.	what said
1	@VirginAmerica plus you've added commercials t...	plus youve added commercials to the experience...
2	@VirginAmerica I didn't today... Must mean I n...	i didnt today must mean i need to take another...
3	@VirginAmerica it's really aggressive to blast...	its really aggressive to blast obnoxious enter...
4	@VirginAmerica and it's a really big bad thing...	and its a really big bad thing about it

Şekil 3 : Metin temizleme fonksiyonu ve çıktısı

```
# tekrar eden satirlari siliyorum
df = df.drop_duplicates()

# eksik degerleri dolduruyorum - silmek yerine anlamlı degerlerle
# negativereason en önemli boş sütun, onu "Belirtilmedi" yapıyorum
df['negativereason'] = df['negativereason'].fillna('Belirtilmedi')

# diğerleri de boş kalsın diye genel bir deger atıyorum
df['airline_sentiment_gold'] = df['airline_sentiment_gold'].fillna('Bilinmiyor')
df['negativereason_gold'] = df['negativereason_gold'].fillna('Bilinmiyor')
df['tweet_coord'] = df['tweet_coord'].fillna('Bilinmiyor')

# tweet_created sütunu string olarak geliyor, datetime a çeviriyorum
df['tweet_created'] = pd.to_datetime(df['tweet_created'])

# sonucu raporlayalım
bitis_kayit = len(df)
print(f"\nTemizleme sonrası toplam kayıt: {bitis_kayit}")
print(f"Silinen tekrarlı kayıt sayısı: {baslangic_kayit - bitis_kayit}")
```

```
Temizleme sonrası toplam kayıt: 14604
Silinen tekrarlı kayıt sayısı: 36
```

Şekil 4 : Duplicate silme ve eksik değer doldurma çıktısı

2.2. Veri Kalitesi Değerlendirmesi

Derste öğrendiğimiz veri kalitesi boyutlarından üçünü bu projede doğrudan uyguladım:

Tamlık (Completeness): Veri setinde özellikle negativereason ve tweet_coord sütunlarında ciddi boşluklar vardı. Bunları silseydim veri setinin neredeyse yarısını kaybederdim. Onun yerine boş değerleri "Belirtilmedi" gibi mantıklı placeholder'larla doldurdum. Sonuçta hiçbir satır kaybetmeden %100 tamlık sağlamış oldum.

Tutarlılık (Consistency): Aynı tweeti birden fazla gösteren tekrarlı kayıtlar vardı. Bunlar analizde sayıları şişirirdi. drop_duplicates() ile bunları temizledim ve veri setindeki her kaydın tekil olmasını sağladım.

Geçerlilik (Validity): tweet_created sütunu metin (string) olarak geliyordu. Bu haliyle tarih bazlı filtreleme veya sıralama yapamazdım. Bunu datetime nesnesine çevirerek geçerli bir formata dönüştürdüm. Ayrıca metin temizleme ile tweetlerdeki URL ve özel karakter kirliliğini giderdim.

AŞAMA 3: MONGODB'YE YÜKLEME VE NOSQL SORGULARI

3.1. MongoDB Şema Tasarımı

Projede SQL yerine MongoDB kullandığım için verileri "Flat Document" (düz doküman) şeklinde modelledim. Her tweet tek bir BSON dokümanı olarak tutuldu. Tweet metni, havayolu, tarih, duygu durumu hepsi aynı dokümanın içinde. Tablolar arası JOIN gerektiren bir ilişki olmadığından bu yapı okuma performansını artırıyor. Zaten NoSQL'in güçlü olduğu taraf da bu: şeması esnek, okuma hızlı.

3.2. Veri Yükleme (Bulk Insert)

Temizlenen veriyi PyMongo ile MongoDB'ye yükledim. Pandas DataFrame'i önce dict listesine çevirip insert_many() ile toplu yükleme (bulk insert) yaptım. Bu sayede 14.000+ doküman tek seferde veritabanına girdi.

```
import pandas as pd
from pymongo import MongoClient

# temizlenmiş csv yi okuyorum
df = pd.read_csv('../data/processed/cleaned_tweets.csv')

# mongodb ye bağlanma
try:
    client = MongoClient('mongodb://localhost:27017/')
    print("MongoDB bağlantısı başarılı!")

    # veritabanı ve koleksiyon
    db = client['twitter_db_pipeline']
    collection = db['tweetler']

    # eğer önceden veri varsa temizleyelim ki çift kayıt olmasın
    collection.delete_many({})

    # pandas dataframe i dict listesine çevirip toplu yükleme (bulk insert)
    records = df.to_dict('records')
    collection.insert_many(records)
    print(f"{len(records)} doküman MongoDB'ye yüklendi.")

except Exception as e:
    print(f"Hata: {e}")
```

✓ 0.4s

MongoDB bağlantısı başarılı!
14604 doküman MongoDB'ye yüklendi.

Şekil 5 : MongoDB bağlantısı ve bulk insert çıktısı

3.3. Sorgular (Toplam 12 İşlem)

Yönergedeki tabloya uygun olarak 12 farklı MongoDB işlemi gerçekleştirdim:

find() Sorguları (3 adet):

- Sorgu 1: \$and operatörü ile çoklu koşul kurdum — hem Virgin America, hem pozitif, hem de güven skoru tam 1.0 olan tweetleri buldum. Projeksiyon ile sadece gerekli alanları döndürdüm ve retweet sayısına göre sıraladım.
- Sorgu 2: \$or operatörü ile rötör veya iptal yaşayan yolcuların tweetlerini buldum. Üstüne güven skoru 0.8'den büyük olanları filtreledim.
- Sorgu 3: \$ne ve \$exists birleşimi ile gerçekten konum bilgisi paylaşmış kullanıcıların negatif tweetlerini getirdim. skip/limit ile sayfalama uyguladım.

```

print("--- 1. find() Sorgulari ---")

# sorgu 1: $and ile coklu kosul - hem pozitif hem guven skoru 1.0 olan virgin america tweetleri
# sadece text ve confidence geri donecek (projeksiyon)
print("\nSorgu 1: Virgin America - pozitif VE guven=1.0 (coklu kosul + projeksiyon):")
sonuc1 = collection.find(
    {"$and": [
        {"airline": "Virgin America"},
        {"airline_sentiment": "positive"},
        {"airline_sentiment_confidence": 1.0}
    ]},
    {"text": 1, "airline_sentiment_confidence": 1, "_id": 0}
).sort("retweet_count", -1).limit(3)
for doc in sonuc1:
    print(doc)

# sorgu 2: $or kullanimi - ya rotar ya iptal yasayan ve guven skoru 0.8 ustu olanlar
print("\nSorgu 2: Rotar VEYA iptal + guven > 0.8 ($or + $gt):")
sonuc2 = collection.find(
    {
        "$or": [
            {"negativereason": "Late Flight"},
            {"negativereason": "Cancelled Flight"}
        ],
        "airline_sentiment_confidence": {"$gt": 0.8}
    },
    {"airline": 1, "negativereason": 1, "airline_sentiment_confidence": 1, "_id": 0}
).limit(3)
for doc in sonuc2:
    print(doc)

# sorgu 3: $ne ve $exists birlesimi - konum bilgisi "Bilinmiyor" olmayan,
# yani gercekten konum paylasan kullanicilarin tweetleri + skip ile sayfalama
print("\nSorgu 3: Konum paylasan kullanicilar ($ne + skip/limit sayfalama):")
sonuc3 = collection.find(
    {
        "tweet_location": {"$ne": "Bilinmiyor", "$exists": True},
        "airline_sentiment": "negative"
    },
    {"name": 1, "tweet_location": 1, "airline": 1, "_id": 0}
).skip(10).limit(3)
for doc in sonuc3:
    print(doc)

```

0.0s

--- 1. find() Sorgulari ---

Sorgu 1: Virgin America - pozitif VE guven=1.0 (coklu kosul + projeksiyon):

```

{'airline_sentiment_confidence': 1.0, 'text': '@VirginAmerica Flying LAX to SFO and after looking at the awesome movie lineup I actually wish I was on a long haul.'}
{'airline_sentiment_confidence': 1.0, 'text': 'Always have it together!!! You're welcome! RT @VirginAmerica: @jessicajaymes You're so welcome.'}
{'airline_sentiment_confidence': 1.0, 'text': '@VirginAmerica thanks for gate checking my baggage on your full flight dfw-lax 883 and giving me early boarding too #sweet'}

```

Sorgu 2: Rotar VEYA iptal + guven > 0.8 (\$or + \$gt):

```

{'airline_sentiment_confidence': 1.0, 'negativereason': 'Late Flight', 'airline': 'Virgin America'}
{'airline_sentiment_confidence': 1.0, 'negativereason': 'Cancelled Flight', 'airline': 'Virgin America'}
{'airline_sentiment_confidence': 1.0, 'negativereason': 'Late Flight', 'airline': 'Virgin America'}

```

Sorgu 3: Konum paylasan kullanicilar (\$ne + skip/limit sayfalama):

```

{'airline': 'Virgin America', 'name': 'jordanpichler', 'tweet_location': nan}
{'airline': 'Virgin America', 'name': 'Cuschoolie1', 'tweet_location': 'Washington DC'}
{'airline': 'Virgin America', 'name': 'amanduhmccarty', 'tweet_location': nan}

```

Şekil 6-7 : find() sorguları çıktıları

Aggregation Pipeline (3 adet):

- Sorgu 4: \$group içinde aynı anda \$sum, \$avg, \$min ve \$max kullanarak her havayolu için toplam tweet, ortalama güven, minimum güven ve maksimum RT sayısını tek sorguda hesapladım.
- Sorgu 5: Negatif tweetleri \$match ile filtreledim, havayoluna göre grupladım, \$project ile \$cond kullanarak ortalama güveni 0.9'un üstündeyse "yuksek", altındaysa "dusuk" olarak etiketledim.
- Sorgu 6: \$addFields ile pipeline içinde yeni alan oluşturdum. Şikayet sebeplerini grupladıktan sonra her birinin toplam içindeki yüzdesini hesapladım.

```

print("--- 2. Aggregation Pipeline Sorgulari ---")

# sorgu 4: her havayolu icin ayni anda birden fazla istatistik hesapla
# $group icinde $sum, $avg, $min, $max bir arada
print("\nSorgu 4: Havayolu bazli detayli istatistikler (sum + avg + min + max):")
pipeline1 = [
    {"$group": {
        "_id": "$airline",
        "toplaml_tweet": {"$sum": 1},
        "ort_guven": {"$avg": "$airline_sentiment_confidence"},
        "min_guven": {"$min": "$airline_sentiment_confidence"},
        "max_rt": {"$max": "$retweet_count"}
    }},
    {"$sort": {"toplaml_tweet": -1}}
]
for doc in collection.aggregate(pipeline1):
    print(doc)

# sorgu 5: $match + $group + $project + $cond - negatif tweetleri al,
# havayoluna gore grupla, eger ortalama guven 0.9 ustuyse "yuksek" degilse "dusuk" etiketle
print("\nSorgu 5: Negatif tweetlerde havayolu bazli guven seviyesi etiketi ($cond):")
pipeline2 = [
    {"$match": {"airline_sentiment": "negative"}},
    {"$group": {
        "_id": "$airline",
        "negatif_sayisi": {"$sum": 1},
        "ort_guven": {"$avg": "$airline_sentiment_confidence"}
    }},
    {"$project": {
        "negatif_sayisi": 1,
        "ort_guven": {"$round": ["$ort_guven", 3]},
        "guven_seviyesi": {
            "$cond": {
                "if": {"$gte": ["$ort_guven", 0.9]},
                "then": "yuksek",
                "else": "dusuk"
            }
        }
    }},
    {"$sort": {"negatif_sayisi": -1}}
]
for doc in collection.aggregate(pipeline2):
    print(doc)

```

```
# sorgu 6: $addFields ile yeni alan ekleyerek pipeline icinde veri zenginlestirme
# sikayet sebeplerine gore grupla ve toplam icindeki yuzdelerini hesapla
print("\nSorgu 6: Sikayet sebeplerinin yuzdesel dagilimi ($addFields):")
toplam_negatif = collection.count_documents({"airline_sentiment": "negative"})
pipeline3 = [
    {"$match": {"airline_sentiment": "negative", "negativereason": {"$ne": "Belirtilmedi"}}},
    {"$group": {"_id": "$negativereason", "adet": {"$sum": 1}}},
    {"$addFields": {
        "yuzde": {
            "$round": [{"$multiply": [{"$divide": ["$adet", toplam_negatif]}, 100]}, 1]
        }
    }},
    {"$sort": {"adet": -1}},
    {"$limit": 5}
]
for doc in collection.aggregate(pipeline3):
    print(doc)
```

✓ 0.0s

--- 2. Aggregation Pipeline Sorgulari ---

Sorgu 4: Havayolu bazli detayli istatistikler (sum + avg + min + max):

```
{'_id': 'United', 'toplam_tweet': 3822, 'ort_guven': 0.9008776818419676, 'min_guven': 0.335, 'max_rt': 7}
{'_id': 'US Airways', 'toplam_tweet': 2913, 'ort_guven': 0.9215784414692757, 'min_guven': 0.34, 'max_rt': 44}
{'_id': 'American', 'toplam_tweet': 2723, 'ort_guven': 0.9162592728608153, 'min_guven': 0.3367, 'max_rt': 5}
{'_id': 'Southwest', 'toplam_tweet': 2420, 'ort_guven': 0.8865159504132231, 'min_guven': 0.3353, 'max_rt': 22}
{'_id': 'Delta', 'toplam_tweet': 2222, 'ort_guven': 0.8698782628262827, 'min_guven': 0.3363, 'max_rt': 31}
{'_id': 'Virgin America', 'toplam_tweet': 504, 'ort_guven': 0.8760861111111111, 'min_guven': 0.3482, 'max_rt': 4}
```

Sorgu 5: Negatif tweetlerde havayolu bazli guven seviyesi etiketi (\$cond):

```
{'_id': 'United', 'negatif_sayisi': 2633, 'ort_guven': 0.933, 'guven_seviyesi': 'yuksek'}
{'_id': 'US Airways', 'negatif_sayisi': 2263, 'ort_guven': 0.946, 'guven_seviyesi': 'yuksek'}
{'_id': 'American', 'negatif_sayisi': 1941, 'ort_guven': 0.944, 'guven_seviyesi': 'yuksek'}
{'_id': 'Southwest', 'negatif_sayisi': 1186, 'ort_guven': 0.921, 'guven_seviyesi': 'yuksek'}
{'_id': 'Delta', 'negatif_sayisi': 955, 'ort_guven': 0.902, 'guven_seviyesi': 'yuksek'}
{'_id': 'Virgin America', 'negatif_sayisi': 181, 'ort_guven': 0.902, 'guven_seviyesi': 'yuksek'}
```

Sorgu 6: Sikayet sebeplerinin yuzdesel dagilimi (\$addFields):

```
{'_id': 'Customer Service Issue', 'adet': 2904, 'yuzde': 31.7}
{'_id': 'Late Flight', 'adet': 1660, 'yuzde': 18.1}
{'_id': 'Can't Tell', 'adet': 1190, 'yuzde': 13.0}
{'_id': 'Cancelled Flight', 'adet': 843, 'yuzde': 9.2}
{'_id': 'Lost Luggage', 'adet': 721, 'yuzde': 7.9}
```

Şekil 8-9 : Aggregation pipeline sorguları çıktıları

İndeks İşlemi (1 adet):

- Sorgu 7: Tekli indeks yerine compound (bileşik) indeks oluşturdum — airline ve airline_sentiment alanlarını birlikte indeksleyerek ikisini aynı anda filtreleyen sorguları hızlandırdım. explain() ile doğruladım.

Update İşlemleri (2 adet):

- Sorgu 8: updateOne() ile RT sayısı 10'un üstündeki bir tweeti bulup \$set ile birden fazla alan ekledim (one_cikan, kategori, inceleme_notu).
- Sorgu 9: updateMany() ile \$set ve \$inc operatörlerini aynı anda kullandım — güven skoru 1.0 olan kayıtlara "kesin_sonuc: true" eklerken aynı zamanda retweet_count değerini 1 artırdım.

Delete İşlemi (1 adet):

- Sorgu 10: \$and ile çoklu koşulda silme yaptım — güven skoru 0.5'in altında VE sebebi "Can't Tell" olan belirsiz kayıtları temizledim.


```

print("--- 3. Indeks, Update ve Delete İşlemleri ---")

# sorgu 7: compound (bilesik) indeks olustur - airline + sentiment birlikte
# bu sayede ikisini birden filtreleyen sorgular cok hizlanir
print("\nSorgu 7: Bilesik indeks (airline + sentiment) olusturuluyor...")
collection.create_index([("airline", 1), ("airline_sentiment", 1)])

# indeksin gercekten kullanildigini explain ile dogrulayalim
aciklama = collection.find(
    {"airline": "Delta", "airline_sentiment": "negative"}
).explain()
kazanan_plan = aciklama['queryPlanner']['winningPlan']
# indeks bilgisi inputStage icinde olabilir
if 'inputStage' in kazanan_plan:
    print("Kullanilan indeks:", kazanan_plan['inputStage'].get('indexName', 'bulunamadi'))
else:
    print("Plan detayi:", kazanan_plan)

# sorgu 8: updateOne - $set ile birden fazla alan ekle + kosullu guncelleme
# en cok RT alan tweeti bul ve ozel olarak isaretle
print("\nSorgu 8: En cok RT alan tweete ozel etiket ($set - coklu alan):")
collection.update_one(
    {"retweet_count": {"$gt": 10}},
    {"$set": {
        "one_cikan": True,
        "kategori": "viral_tweet",
        "inceleme_notu": "yuksek etkilesim"
    }}
)
kontrol = collection.find_one({"one_cikan": True}, {"text": 1, "retweet_count": 1, "kategori": 1, "_id": 0})
print("Guncellenen kayit:", kontrol)

# sorgu 9: updateMany - $set + $inc birlikte kullanimi
# guven skoru 1.0 olan tum tweetlere "kesin_sonuc: true" ekle
# ayni zamanda retweet_count degerini 1 artir ($inc)
print("\nSorgu 9: Guven=1.0 olanlara alan ekle + RT sayisini 1 artir ($set + $inc):")
guncelleme = collection.update_many(
    {"airline_sentiment_confidence": 1.0},
    {
        "$set": {"kesin_sonuc": True},
        "$inc": {"retweet_count": 1}
    }
)
print(f"Guncellenen kayit sayisi: {guncelleme.modified_count}")

```

```

# sorgu 10: deleteMany - karmasik kosulla silme
# guven skoru 0.5 in altinda VE sebebi "Can't Tell" olan kayitlar analize zarar veriyor, silelim
print("\nSorgu 10: Dusuk guvenli + belirsiz sebepli kayitlari sil ($lt + $and):")
silinen = collection.delete_many({
    "$and": [
        {"airline_sentiment_confidence": {"$lt": 0.5}},
        {"negative_reason": "Can't Tell"}
    ]
})
print(f"Silinen kayit sayisi: {silinen.deleted_count}")

```

✓ 0.3s

--- 3. Indeks, Update ve Delete İşlemleri ---

Sorgu 7: Bilesik indeks (airline + sentiment) olusturuluyor...

Kullanilan indeks: airline_1_airline_sentiment_1

Sorgu 8: En cok RT alan tweete ozel etiket (\$set - coklu alan):

Guncellenen kayit: {'retweet_count': 22, 'text': '@SouthwestAir beautiful day in Seattle! <http://t.co/igu0PPVq2S>', 'kategori': 'viral_tweet'}

Sorgu 9: Guven=1.0 olanlara alan ekle + RT sayisini 1 artir (\$set + \$inc):

Guncellenen kayit sayisi: 10409

Sorgu 10: Dusuk guvenli + belirsiz sebepli kayitlari sil (\$lt + \$and):

Silinen kayit sayisi: 25

Şekil 10-11 : İndeks, update ve delete sorguları çıktıları

İleri Seviye Sorgular (2 adet):

- Sorgu 11: Gelişmiş regex ile "cancel" veya "delay" kelimelerini tek regex'te pipe (|) operatörüyle yakaladım. Üstüne \$in ile sadece United ve Delta havayollarını filtreledim. Yani regex + \$in birleşik kullanım.
- Sorgu 12: \$in operatörünü aggregation pipeline içinde kullandım. Büyük 3 havayolunu seçip, her birinin duygu dağılımını (pozitif/negatif/nötr sayılarını) çıkardım — \$group'ta bileşik _id kullanarak havayolu + duygu çiftini gruplayarak.

```
print("--- 4. İleri Seviye Sorgular ---")

# sorgu 11: gelişmiş regex - "cancel" VEYA "delay" içeren tweetleri bul
# iki kelimeyi tek regex ile yakalıyorum (pipe | operatörü)
# ayrıca sadece United ve Delta için filtrele ($in ile birleşik kullanım)
print("\nSorgu 11: 'cancel' veya 'delay' içeren United/Delta tweetleri (Regex + $in):")
sonuc11 = collection.find(
    {
        "text": {"$regex": "cancel|delay", "$options": "i"},
        "airline": {"$in": ["United", "Delta"]}
    },
    {"airline": 1, "text": 1, "negativeReason": 1, "_id": 0}
).sort("airline_sentiment_confidence", -1).limit(3)
for doc in sonuc11:
    print(doc)

# sorgu 12: $in ile aggregation pipeline birlesimi
# sadece büyük 3 havayolunu al, her birinin duygu dagilimini (pos/neg/neutral sayilarini) cikar
print("\nSorgu 12: Büyük 3 havayolunun duygu dagilimi ($in + aggregation):")
pipeline_ileri = [
    {"$match": {"airline": {"$in": ["United", "American", "US Airways"]}}},
    {"$group": {
        "_id": {"havayolu": "$airline", "duygu": "$airline_sentiment"},
        "adet": {"$sum": 1}
    }},
    {"$sort": {"_id.havayolu": 1, "adet": -1}}
]
for doc in collection.aggregate(pipeline_ileri):
    print(doc)

✓ 0.0s

--- 4. İleri Seviye Sorgular ---

Sorgu 11: 'cancel' veya 'delay' içeren United/Delta tweetleri (Regex + $in):
{'negativeReason': 'Cancelled Flight', 'airline': 'Delta', 'text': '@JetBlue but by Cancelled Flighting my flight and pushing me to the next day I'd lose $150 hotel which was why I was trying to get a same-day flight.'}
{'negativeReason': 'Cancelled Flight', 'airline': 'Delta', 'text': '@JetBlue Cancelled Flighted my flight. Went with another airline 2 leave 2day. They Cancelled Flighted also. Called JetBlue & got same flight but now $250 more!'}
{'negativeReason': 'Late Flight', 'airline': 'Delta', 'text': '@JetBlue why was Flight 1856 delayed to Buffalo ? It's a direct flight and the plane is at the gate.'}

Sorgu 12: Büyük 3 havayolunun duygu dagilimi ($in + aggregation):
{'_id': {'havayolu': 'American', 'duygu': 'negative'}, 'adet': 1941}
{'_id': {'havayolu': 'American', 'duygu': 'neutral'}, 'adet': 455}
{'_id': {'havayolu': 'American', 'duygu': 'positive'}, 'adet': 327}
{'_id': {'havayolu': 'US Airways', 'duygu': 'negative'}, 'adet': 2259}
{'_id': {'havayolu': 'US Airways', 'duygu': 'neutral'}, 'adet': 381}
{'_id': {'havayolu': 'US Airways', 'duygu': 'positive'}, 'adet': 269}
{'_id': {'havayolu': 'United', 'duygu': 'negative'}, 'adet': 2625}
{'_id': {'havayolu': 'United', 'duygu': 'neutral'}, 'adet': 697}
{'_id': {'havayolu': 'United', 'duygu': 'positive'}, 'adet': 492}
```

Şekil 12 : Regex ve \$in + aggregation çıktıları

AŞAMA 4: PYSPARK İLE DAĞITIK VERİ İŞLEME

Temizlenmiş veri setini PySpark DataFrame'e yükledim ve büyük veri işleme mantığıyla çeşitli operasyonlar uyguladım. Toplamda 8 işlem gerçekleştirdim.

4.1. Dönüşüm (Transformation) İşlemleri — 3 adet

- İşlem 1 (filter + select): isin() ile birden fazla havayolunu, ardından negatif duygu ve güven skoru 0.9 üstü koşullarını zincirleme filtreledim. Basit bir filtre yerine üç koşulu aynı anda uyguladım.
- İşlem 2 (withColumn + when/otherwise): Güven skoruna göre koşullu sütun oluşturdum — 1.0 ise "kesin", 0.8-1.0 arası "yuksek", altıysa "dusuk" etiketini veren bir mantık kurdum. Bu Spark'ın SQL'deki CASE WHEN karşılığı.
- İşlem 3 (groupBy + multiple agg): Havayollarına göre aynı anda tweet sayısı, ortalama güven ve ortalama RT sayısını hesapladım. Tek groupBy'da birden fazla aggregation fonksiyonu kullandım.

Bu üç işlem de Spark'ın lazy evaluation (tembel değerlendirme) prensibiyle çalışıyor; yani bir action tetiklenene kadar gerçekte çalışmıyorlar, sadece plan oluşturluyorlar.

```
print("--- 1. Donusum (Transformation) Islemleri ---")

# islem 1: karmasik filter + select
# United veya American havayoluna ait, negatif, guven skoru 0.9 ustu tweetler
print("\nIslem 1: United/American negatif + guven>0.9 (complex filter & select):")
filtreli = spark_df.filter(
    (col("airline").isin("United", "American")) &
    (col("airline_sentiment") == "negative") &
    (col("airline_sentiment_confidence") > 0.9)
).select("airline", "negativereason", "airline_sentiment_confidence", "text")
filtreli.show(3, truncate=50)

# islem 2: withColumn + when/otherwise - guven skoruna gore kategori sutunu olustur
# 1.0 ise "kesin", 0.8-1.0 arasi "yuksek", altiysa "dusuk" diye etiketle
print("\nIslem 2: Guven kategorisi sutunu (withColumn + when/otherwise):")
df_kategorili = spark_df.withColumn(
    "guven_kategori",
    when(col("airline_sentiment_confidence") == 1.0, "kesin")
    .when(col("airline_sentiment_confidence") >= 0.8, "yuksek")
    .otherwise("dusuk")
)
df_kategorili.select("airline", "airline_sentiment_confidence", "guven_kategori").show(5)

# islem 3: groupBy + birden fazla aggregation fonksiyonu ayni anda
# havayoluna gore tweet sayisi, ortalama guven, max retweet
print("\nIslem 3: Havayolu bazli coklu istatistik (groupBy + multiple agg):")
coklu_agg = spark_df.groupBy("airline").agg(
    count("*").alias("tweet_sayisi"),
    spark_round(avg("airline_sentiment_confidence"), 3).alias("ort_guven"),
    avg("retweet_count").alias("ort_rt")
).orderBy(col("tweet_sayisi").desc())
coklu_agg.show()
```

--- 1. Donusum (Transformation) Islemleri ---

Islem 1: United/American negatif + guven>0.9 (complex filter & select):

airline	negativereason	airline_sentiment_confidence	text
United	Cancelled Flight	1.0	@united still no refund or word via DM. Please ...
United	Late Flight	1.0	@united Delayed due to lack of crew and now del...
United	Flight Booking Problems	1.0	@united is the worst. Nonrefundable First class...

only showing top 3 rows

Islem 2: Guven kategorisi sutunu (withColumn + when/otherwise):

airline	airline_sentiment_confidence	guven_kategori
Virgin America	1.0	kesin
Virgin America	0.3486	dusuk
Virgin America	0.6837	dusuk
Virgin America	1.0	kesin
Virgin America	1.0	kesin

only showing top 5 rows

Islem 3: Havayolu bazli coklu istatistik (groupBy + multiple agg):

airline	tweet_sayisi	ort_guven	ort_rt
United	3822	0.901	0.11015175300889586
US Airways	2913	0.922	0.0854788877445932
American	2723	0.916	0.0426000734484025
Southwest	2420	0.887	0.05991735537190083
Delta	2222	0.87	0.11341134113411341
Virgin America	504	0.876	0.051587301587301584

Şekil 13-14 : Transformation çıktıları

4.2. Eylem (Action) İşlemleri — 2 adet

- İşlem 4 (count + distinct): Sadece toplam kayıt değil, aynı zamanda benzersiz kullanıcı sayısını ve benzersiz konum sayısını da hesapladım. distinct().count() ile farklı değerlerin sayısını aldım.
- İşlem 5 (describe): Tek sütun yerine hem güven skoru hem retweet sayısı için aynı anda istatistik çıkardım.

```
print("--- 2. Eylem (Action) Islemleri ---")

# islem 4: count + distinct count birlikte
toplam = spark_df.count()
benzersiz_kullanici = spark_df.select("name").distinct().count()
benzersiz_konum = spark_df.filter(col("tweet_location") != "Bilinmiyor").select("tweet_location").distinct().count()
print(f"Toplam kayıt: {toplam}")
print(f"Benzersiz kullanıcı: {benzersiz_kullanici}")
print(f"Benzersiz konum (bilinmiyor haric): {benzersiz_konum}\n")

# islem 5: describe - birden fazla numerik sutun icin istatistik
print("Birden fazla sutun icin istatistik (describe):")
spark_df.describe(['airline_sentiment_confidence', 'retweet_count']).show()
```

--- 2. Eylem (Action) Islemleri ---

Toplam kayıt: 14604
Benzersiz kullanıcı: 7701
Benzersiz konum (bilinmiyor haric): 3081

Birden fazla sutun icin istatistik (describe):

summary	airline_sentiment_confidence	retweet_count
count	14604	14604
mean	0.899922760887427	0.08278553820870994
stddev	0.162954988530412	0.7466472933512464
min	0.335	0
max	1.0	44

Şekil 15 : Action çıktıları

4.3. Spark SQL — 2 adet

Spark DataFrame'i createOrReplaceTempView() ile geçici tabloya çevirip üzerinde SQL yazdım:

- İşlem 6: CASE WHEN ile her havayolunun negatif, pozitif ve nötr tweet sayılarını ayrı sütunlarda hesapladım. HAVING ile sadece 500'den fazla tweeti olan havayollarını gösterdim.
- İşlem 7: Subquery (iç içe sorgu) kullandım. Önce her havayolunun negatif tweet oranını hesapladım, sonra dış sorguda genel ortalamadan kötü olanları filtreledim. Yani "ortalamanın altında kalan havayolları" sorusunu SQL ile cevapladım.

```
print("--- 3. Spark SQL İşlemleri ---")

# geçici tablo oluşturun
spark_df.createOrReplaceTempView('tweets_table')

# işlem 6: CASE WHEN ile duygu etiketine göre türkçe açıklama sütunu oluşturun
# + GROUP BY + HAVING (sadece 1000'den fazla tweeti olan havayollarını göster)
print("\nSQL Sorgu 1: CASE WHEN + HAVING - büyük havayollarının duygu dağılımı:")
sql1 = spark.sql("""
SELECT
    airline,
    SUM(CASE WHEN airline_sentiment = 'negative' THEN 1 ELSE 0 END) as negatif,
    SUM(CASE WHEN airline_sentiment = 'positive' THEN 1 ELSE 0 END) as pozitif,
    SUM(CASE WHEN airline_sentiment = 'neutral' THEN 1 ELSE 0 END) as notr,
    COUNT(*) as toplam
FROM tweets_table
GROUP BY airline
HAVING COUNT(*) > 500
ORDER BY negatif DESC
""")
sql1.show()

# işlem 7: iç içe sorgu (subquery) - her havayolunun negatif tweet oranını hesapla
# ve genel ortalamadan kötü olanları bul
print("\nSQL Sorgu 2: Negatif tweet oranı genel ortalamanın üstünde olan havayolları (subquery):")
sql2 = spark.sql("""
SELECT
    airline,
    ROUND(negatif_oran, 2) as negatif_yuzde
FROM (
    SELECT
        airline,
        (SUM(CASE WHEN airline_sentiment = 'negative' THEN 1 ELSE 0 END) * 100.0 / COUNT(*)) as negatif_oran
    FROM tweets_table
    GROUP BY airline
) alt_sorgu
WHERE negatif_oran > (
    SELECT SUM(CASE WHEN airline_sentiment = 'negative' THEN 1 ELSE 0 END) * 100.0 / COUNT(*)
    FROM tweets_table
)
ORDER BY negatif_yuzde DESC
""")
sql2.show()
```

```
--- 3. Spark SQL İşlemleri ---

SQL Sorgu 1: CASE WHEN + HAVING - büyük havayollarının duygu dağılımı:
+-----+-----+-----+-----+-----+
| airline|negatif|pozitif|notr|toplam|
+-----+-----+-----+-----+-----+
| United| 2633| 492| 697| 3822|
| US Airways| 2263| 269| 381| 2913|
| American| 1941| 327| 455| 2723|
| Southwest| 1186| 570| 664| 2420|
| Delta| 955| 544| 723| 2222|
| Virgin America| 181| 152| 171| 504|
+-----+-----+-----+-----+-----+

SQL Sorgu 2: Negatif tweet oranı genel ortalamanın üstünde olan havayolları (subquery):
+-----+-----+
| airline|negatif_yuzde|
+-----+-----+
| US Airways| 77.69|
| American| 71.28|
| United| 68.89|
+-----+-----+
```

Şekil 16-17 : Spark SQL çıktıları

4.4. Pandas vs PySpark Karşılaştırması — 1 adet

Bu sefer basit bir count yerine daha karmaşık bir işlem kıyasladım: negatif tweetleri filtreleme + havayoluna göre gruptlama + hem sayı hem ortalama güven hesaplama. Aynı işlemi Pandas ve PySpark ile yapıp sürelerini ölçtüm.

- İşlem 8: Pandas yaklaşık 0.004 saniyede, PySpark yaklaşık 0.94 saniyede bitti.

Bu sonuç şaşırtıcı değil aslında. Veri setimiz sadece ~14.000 satır ve tek makinede çalışıyoruz. PySpark burada Spark oturumunu başlatma, iş planı oluşturma gibi overhead'ler yüzünden yavaş kalıyor. Ama veri milyonlarca satıra çıktığında Spark dağıtık yapısıyla (birden fazla makineye yayarak) Pandas'ı çok geride bırakır. Küçük veride Pandas, büyük veride PySpark mantığı burada açıkça görülüyor.

```
import pandas as pd
import time

print("--- 4. Pandas vs PySpark Performans Karsilastirmasi ---")

# aynı veriyi pandas ile de okuyalım
pandas_df = pd.read_csv('/content/drive/MyDrive/Buyuk_Veri_Donem_Projesi/twitter_big_data_pipeline/data/processed/clear')

# pandas testi - havayoluna göre negatif tweet sayma + ortalama güven
t1 = time.time()
pandas_sonuc = pandas_df[pandas_df['airline_sentiment'] == 'negative'].groupby('airline').agg(
    adet=('airline_sentiment', 'count'),
    ort_guven=('airline_sentiment_confidence', 'mean')
)
t2 = time.time()
pandas_sure = t2 - t1

# pyspark testi - aynı işlem
t3 = time.time()
spark_sonuc = spark_df.filter(col("airline_sentiment") == "negative") \
    .groupBy("airline") \
    .agg(
        count("*").alias("adet"),
        avg("airline_sentiment_confidence").alias("ort_guven")
    ).collect()
t4 = time.time()
spark_sure = t4 - t3

print(f"Pandas suresi: {pandas_sure:.5f} sn")
print(f"PySpark suresi: {spark_sure:.5f} sn")

print("\nNot: ~14.000 satırlık küçük bir veride Pandas daha hızlı çıkabilir.")
print("PySpark'in asıl gücü milyonlarca satırlık verilerde dağıtık işlemede ortaya çıkar.")

spark.stop()
```

--- 4. Pandas vs PySpark Performans Karsilastirmasi ---
Pandas suresi: 0.02453 sn
PySpark suresi: 0.42851 sn

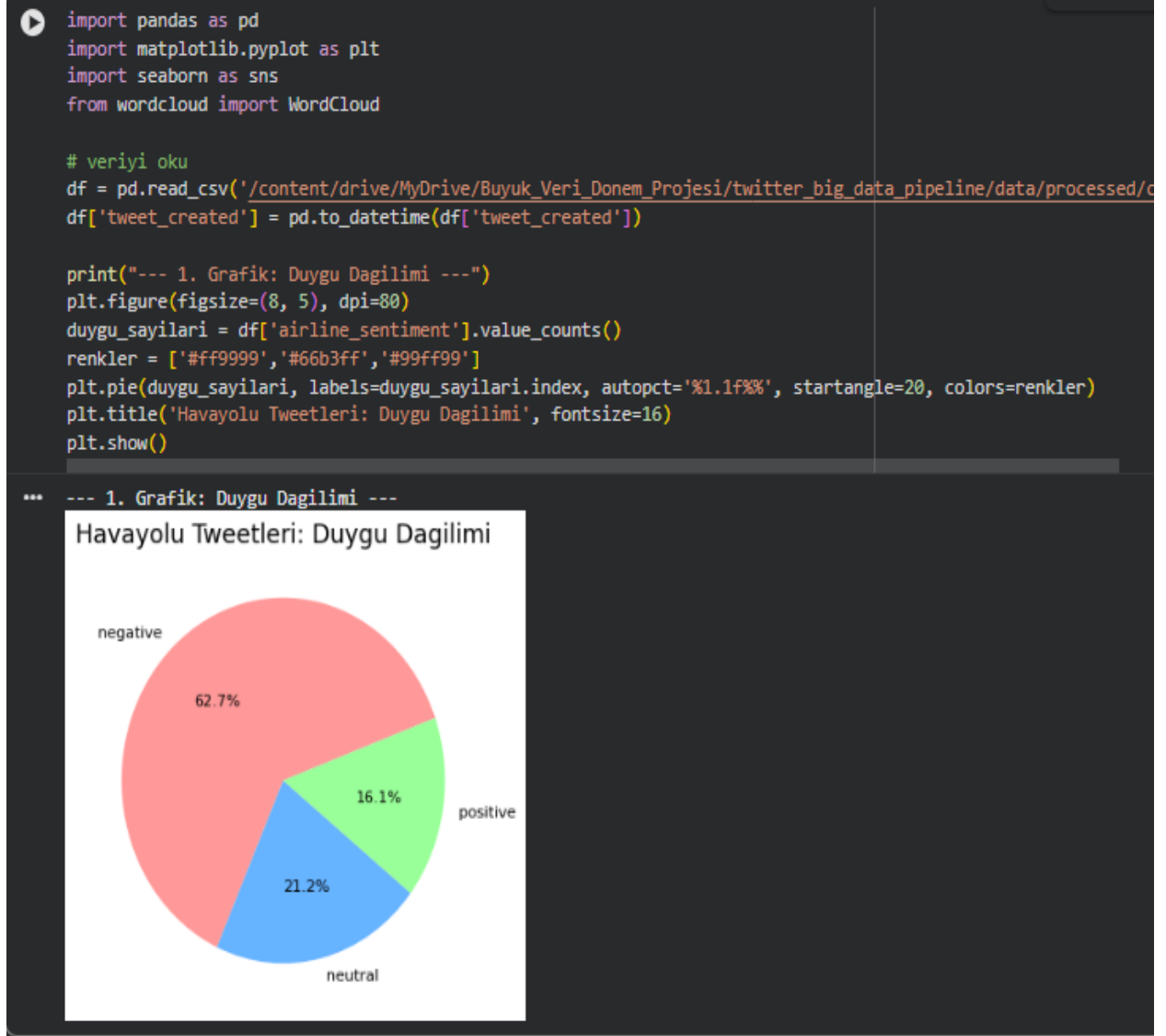
Not: ~14.000 satırlık küçük bir veride Pandas daha hızlı çıkabilir.
PySpark'in asıl gücü milyonlarca satırlık verilerde dağıtık işlemede ortaya çıkar.

Şekil 18 : Performans karşılaştırması çıktısı

AŞAMA 5: VERİ GÖRSELLEŞTİRME VE ANALİZ

Matplotlib, Seaborn ve WordCloud kütüphaneleriyle 5 farklı grafik oluşturdum. Her birinin altında "bu grafik bize ne söylüyor?" sorusuna cevap verdim.

5.1. Duygu Dağılımı (Pie Chart)



Şekil 19 : Pasta grafik

Yorum: Bu grafiğe baktığımda ilk dikkatimi çeken şey tweetlerin büyük çoğunluğunun negatif olması. Yaklaşık %63'ü negatif, %21'i nötr ve sadece %16'sı pozitif. Buradan şunu anlıyorum: insanlar havayoluyla iyi bir deneyim yaşadığında pek tweet atmıyor ama bir sorun yaşadığında hemen Twitter'a koşuyor. Yani bu veri seti doğası gereği şikayet ağırlıklı.

5.2. Kelime Bulutu (WordCloud)

```
print("--- 2. Grafik: Kelime Bulutu (WordCloud) ---")

# tum temizlenmis tweet metinlerini birlestir
tum_metin = " ".join(tweet for tweet in df['cleaned_text'].dropna())

plt.figure(figsize=(10, 6), dpi=100)
wc = WordCloud(width=800, height=400, background_color='white', max_words=100).generate(tum_metin)
plt.imshow(wc, interpolation='bilinear')
plt.axis('off')
plt.title('Tweetlerde En Cok Gecen Kelimeler', fontsize=16)
plt.show()
```

```
*** --- 2. Grafik: Kelime Bulutu (WordCloud) ---
```



Şekil 20 : WordCloud

Yorum: Kelime bulutuna bakınca en büyük yazılan kelimeler "flight", "cancelled", "delay", "service" gibi operasyonel sorunlarla ilgili kelimeler. Pozitif kelimeler (great, love, thank gibi) çok küçük kalıyor. Bu da pasta grafikte tutarlı: insanlar genellikle uçuş iptallerinden ve rötarlardan şikayet ediyor.

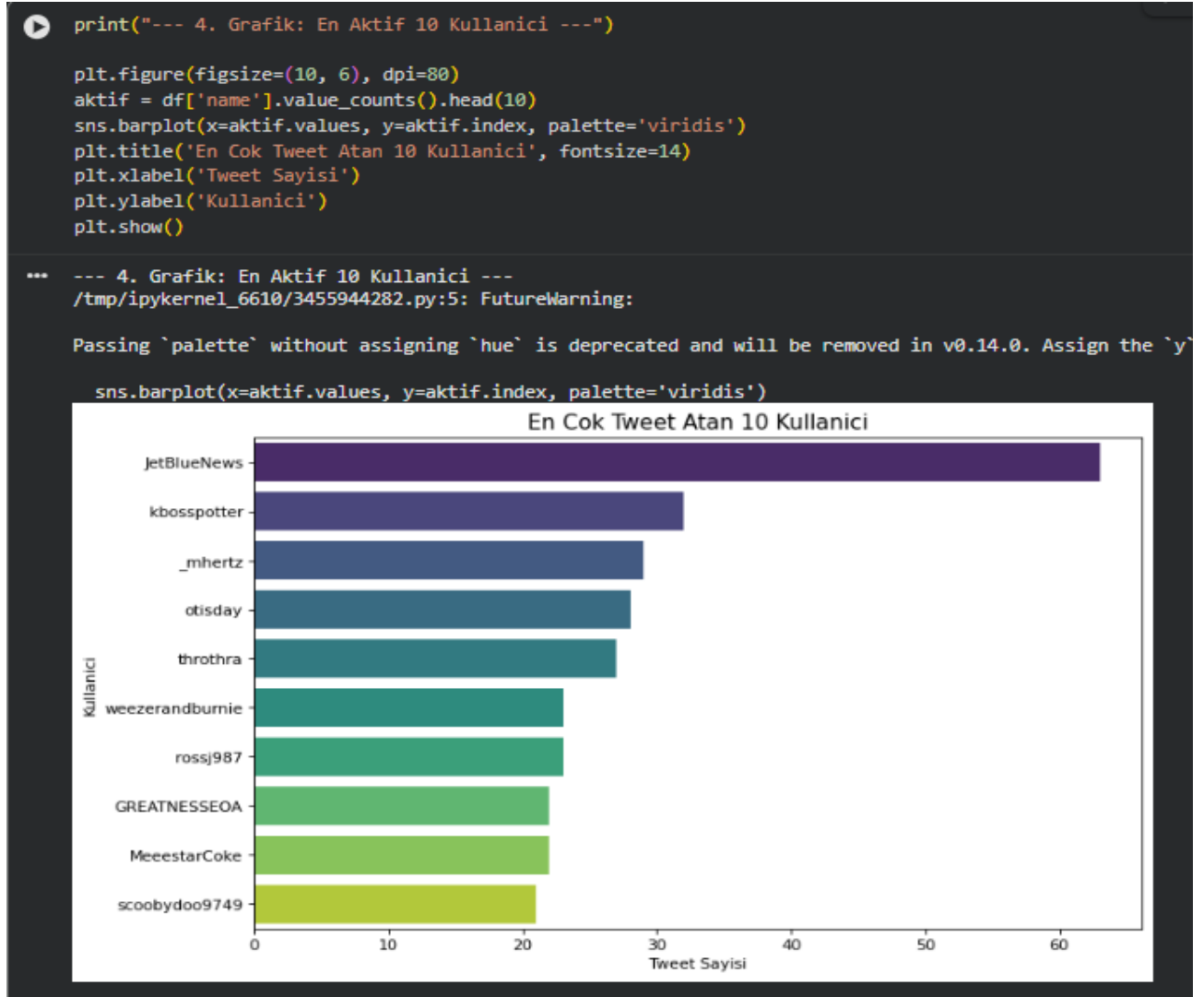
5.3. Zaman Serisi (Line Plot)



Şekil 21 : Zaman serisi grafiği

Yorum: Günlere göre tweet sayısına baktığımda belirli günlerde ani sıçramalar görüyorum. Bu peak noktaları büyük ihtimalle toplu uçuş iptallerinin veya hava koşullarından kaynaklanan gecikmelerin yaşandığı günler. Normal günlerde tweet sayısı göreceli sabit giderken kriz günlerinde bir anda patlıyor. Bir havayolu şirketi bu grafiği izleyerek hangi günlerde sorun yaşadığını net görebilir.

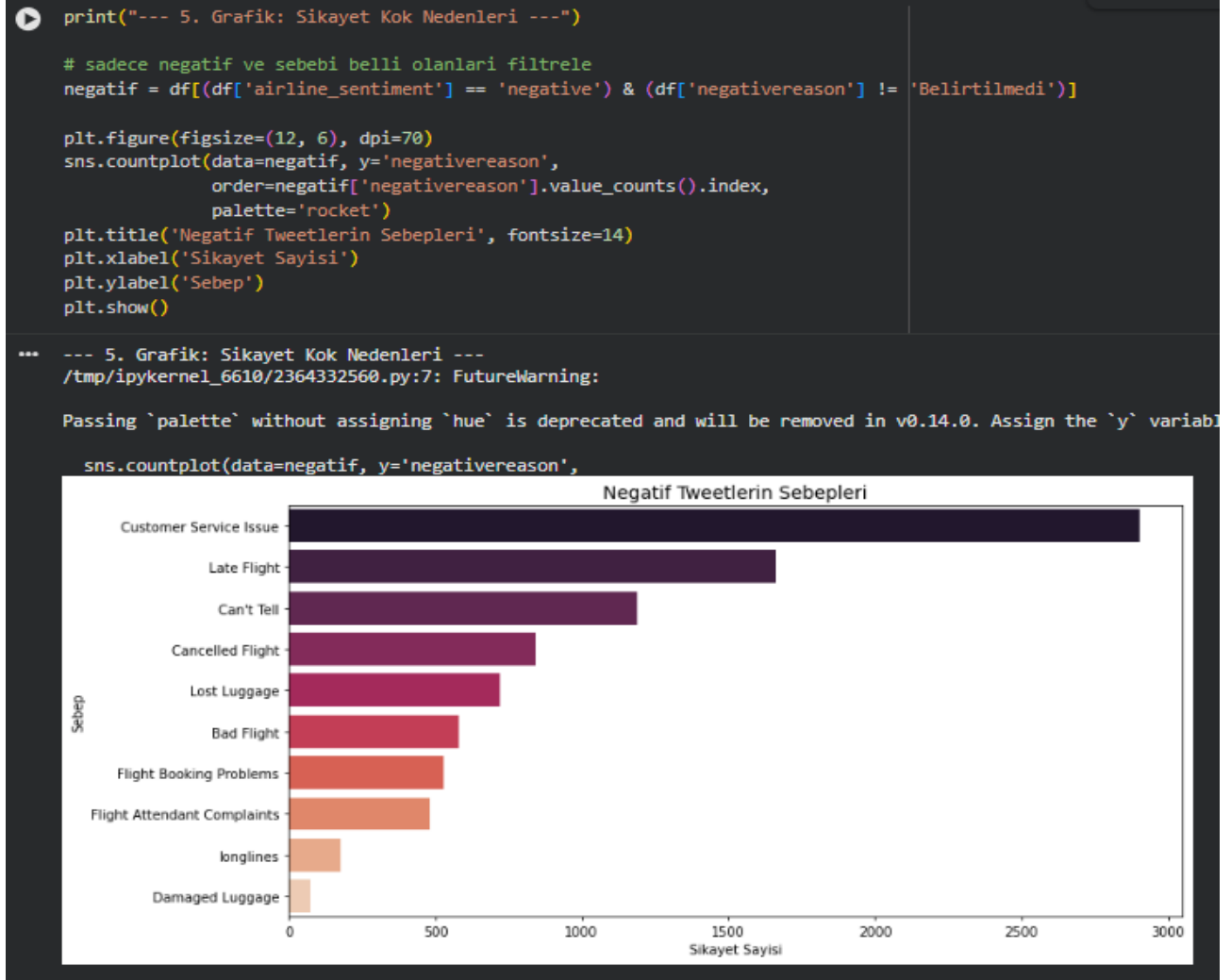
5.4. En Aktif 10 Kullanıcı (Bar Plot)



Şekil 22 : Bar grafiği

Yorum: Bu grafikte bazı kullanıcıların diğerlerinden çok daha fazla tweet attığını görüyorum. Mesela ilk sıradaki kullanıcı tek başına onlarca tweet atmış. Bu kişiler muhtemelen tek seferlik şikayetçi değil, havayoluyla uzun süreli bir sorunu olan (belki kayıp bagaj takibi gibi) ve sürekli etkileşime giren kullanıcılar.

5.5. Şikayetlerin Kök Nedenleri (Horizontal Bar)



Şekil 23 : Kök neden grafiği

Yorum: Bu grafik en önemli bulguyu ortaya koyuyor bence. Müşteri memnuniyetsizliğinin bir numaralı sebebi "Customer Service Issues" yani müşteri hizmetleri sorunu. İkinci sırada "Late Flight" (rötar) var. İlginç olan şu: insanlar rötardan bile çok, muhatap bulamadıklarından şikayet ediyor. Yani uçuş gecikse bile müşteri hizmetleri düzgün bilgi verse belki bu kadar tepki olmayacak. Bir havayolu şirketi bu veriye bakarak öncelikle müşteri hizmetleri kalitesini artırmaya odaklanmalı.

AŞAMA 6: ETL DİYAGRAMI VE SİSTEM MİMARİSİ

6.1. ETL vs. ELT Karşılaştırması

Derste ETL ve ELT kavramlarını işlemiştik. Kısaca hatırlarsak:

ETL (Extract, Transform, Load): Veriyi kaynaktan çek, bir ara ortamda (bellekte veya başka bir sunucuda) temizle/dönüştür, sonra hedef veritabanına temiz haliyle yükle. Klasik veri ambarı yaklaşımı. Veri hedefe ulaşmadan önce şekilleniyor.

ELT (Extract, Load, Transform): Veriyi kaynaktan çek, olduğu gibi hedef sisteme yükle, temizleme/dönüştürme işlemlerini hedef sistemin kendi gücüyle orada yap. Bulut tabanlı veri gölü (data

lake) yaklaşımlarında yaygın. BigQuery, Snowflake gibi sistemler buna uygun çünkü kendi içlerinde güçlü hesaplama kapasiteleri var.

Bu proje hangisine yakın?

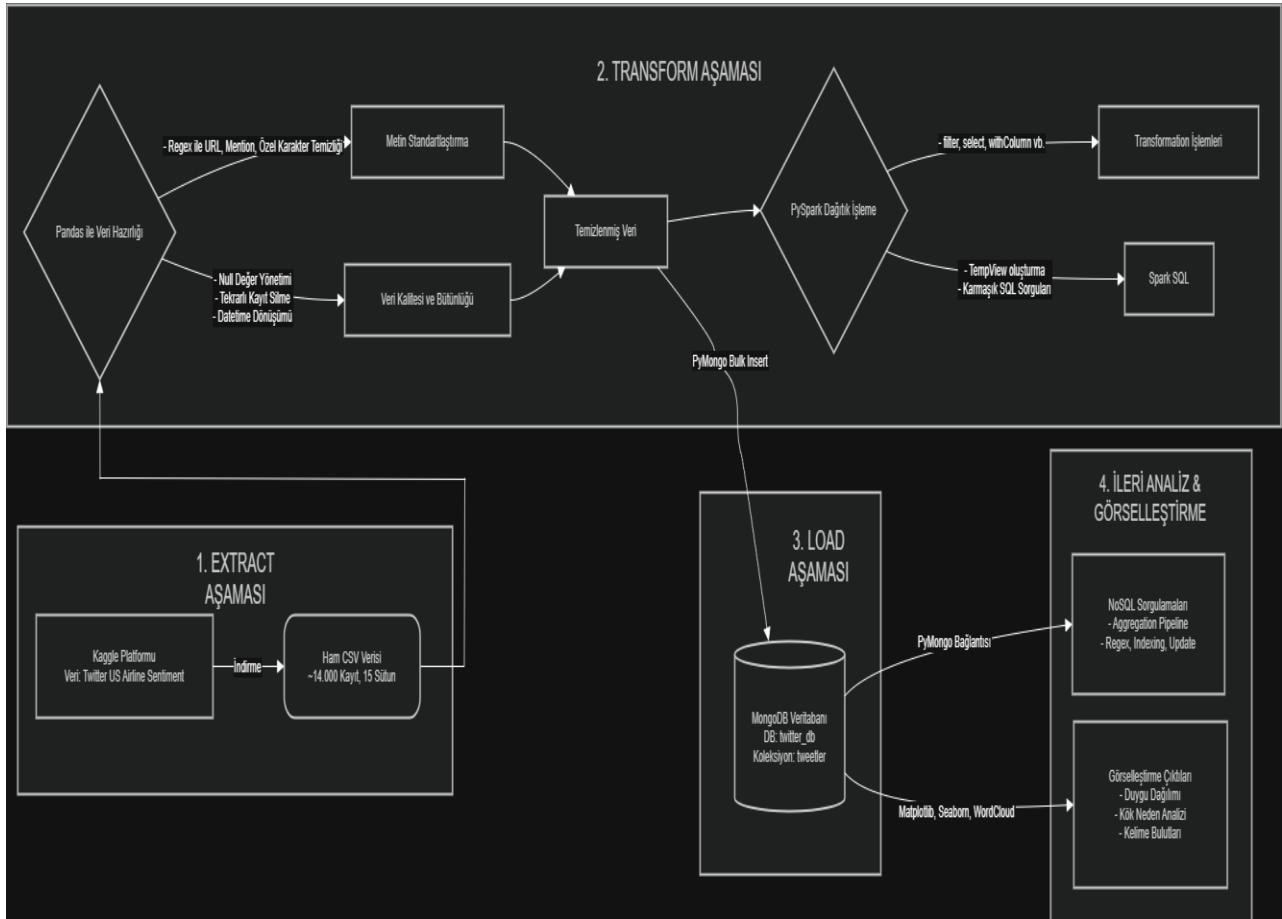
Benim yaptığım akış kesinlikle ETL'e daha yakın. Şöyle ki:

- **Extract:** Veriyi Kaggle'dan CSV olarak indirdim.
- **Transform:** Veriyi doğrudan MongoDB'ye yüklemek yerine önce Python/Pandas ortamında temizledim. URL silme, eksik değer doldurma, duplicate kaldırma gibi işlemleri bellekte yaptım.
- **Load:** Veri tamamen temizlenip hazır olduktan sonra MongoDB'ye toplu yükleme (insert_many) ile aktardım.

Eğer ELT yaklaşımı uygulaysaydım, ham CSV'yi olduğu gibi MongoDB'ye yükler ve temizleme işlemlerini MongoDB'nin aggregation pipeline'ı ile veritabanı içinde yapardım. Ama küçük ölçekli bir projede Pandas ile temizlemek hem daha kolay hem daha kontrollü olduğu için ETL tercih ettim.

6.2. Uçtan Uca Proje Akış Diyagramı

Aşağıdaki diyagram projenin tüm aşamalarını göstermektedir:



Şekil 24 : draw.io ile çizilmiş ETL akış diyagramı

SONUÇ

Bu projede Kaggle'dan aldığım havayolu tweet verisini baştan sona işledim. Pandas ile temizledim, MongoDB'ye yükleyip 12 farklı sorgu yazdım, PySpark ile dağıtık işleme adımlarını uyguladım ve 5 farklı görselleştirme oluşturdum. Proje boyunca derste öğrendiğimiz DIKW piramidi, veri kalitesi boyutları, NoSQL modelleme ve ETL kavramlarını doğrudan pratiğe dökmüş oldum. En dikkat çekici bulgu, havayolu müşterilerinin en çok müşteri hizmetleri konusunda şikayet etmesi ve tweet trafiğinin kriz günlerinde patlama yapması oldu.